BIOT 6900 · AI-POWERED BIOTECH WORKFLOWS

Module 6 Lab Guide, Part 1: Build a Polygenic Risk Score from First Principles

Week 6 · the toy model · instructor-led, not graded · Fall 2026

Notebook: `BIOT6900_Module6_Part1_Toy.ipynb` Runtime: about 20 minutes No data files, no downloads, no network

What this lab is

In the lecture you met GWAS, polygenic risk scores, the AUC, calibration, and the portability problem. The risk, once you start using a PRS, is that it becomes an oracle: you download a set of weights, apply them, and a single number drops out with no way to ask what it is made of or where it fails.

This lab removes that risk by having you build a complete PRS pipeline by hand, a few lines per stage, on a tiny synthetic cohort you fully control. By the end you will have built the score from genotypes and weights, watched cases and controls separate, measured how well it discriminates, and then removed a single variant and watched most of the signal disappear.

Step	What you build	The real thing it stands in for
1	A synthetic case/control cohort	A real GWAS cohort or biobank
2	A set of downloaded weights	A PGS Catalog score (PRS-CS, LDpred2)
3	The PRS itself: dosages × weights, summed	plink --score, an applied PGS
4	A case-vs-control separation plot	The PRS distributions reported in papers
5	Discrimination: the AUC and ROC	Validation metrics
6	A leave-one-locus-out test	The APOE in/out analysis

The one idea to carry through

A PRS is just dosages times weights, summed. Everything that makes it hard lives in the numbers you feed in (which weights, whose genotypes) and the questions you ask of the output (does it separate, is it calibrated, does it transfer), not in the arithmetic.

Maps to course outcomes: LO1 (apply industry-standard tools), LO3 (interpret results in biological and clinical context), LO4 (evaluate model performance and limitations).

Before you start

1 · Your environment

Use the same biot6900 conda environment from Module 1. This notebook needs only numpy, pandas, matplotlib and scikit-learn, all already in it. There is nothing to install and nothing to download. If your environment is broken, come to office hours before class rather than losing the session to setup.

2 · What you do not need

- No data files. The cohort is generated inside the notebook and printed with a clear SYNTHETIC label.
- No internet connection. Nothing is fetched.
- No API keys, no model weights, no GPU.

This is deliberate. Part 1 has no external dependency at all, so nobody is blocked while we work through the mechanics. The real genotypes and real weights arrive in Part 2.

The toy cohort

A PRS fails on bookkeeping as often as on modelling, so the lab starts with a cohort whose every piece is visible. It has everything a real PRS analysis has, at roughly 1/100,000 scale.

Piece	What it is
Genotypes G	4,000 people × 20 variants, each entry a dosage of 0, 1, or 2 risk alleles
True effects	Per-allele log-odds: nineteen tiny, one large (the APOE-like variant, odds ratio e^0.8 ≈ 2.2 per allele)
Weights	The true effects plus a little noise: the GWAS-estimated weights you would download
Disease	A logistic model turns each person's genetic score into a probability; about 33% become cases

Why one variant is large. A real “polygenic” score for some diseases leans heavily on a single locus (APOE in Alzheimer's). The one big effect here is what makes that dominance visible in Step 6.

Do this before running anything

> Predict: when you remove the single large-effect variant in Step 6, will the AUC drop a little or a lot? Write your answer down now. Step 6 checks it.

Walking through the six steps

Step 1 · Build the synthetic cohort

What it does. Draws genotype dosages from each variant's allele frequency, assigns per-allele log-odds effects with one large APOE-like variant, then generates disease status from a logistic model of the genetic score.

What you should see. A printed line: 4,000 people, 20 variants, about 33% cases.

The point. Because you generated it, you know the truth. Nothing downstream is a mystery; every number can be traced back to this cell. There is no machine learning here, just a model of how genotypes relate to disease.

Step 2 · The weights you would download

What it does. Builds the weights table (variant, risk-allele frequency, weight) as the true effects plus a little noise.

What you should see. A 20-row table of per-allele log-odds, with the APOE-like variant's weight far larger than the rest.

The point. In the real world you never know the true effects; you download vetted estimates from the PGS Catalog. The noise stands in for GWAS estimation error, which is why real weights are imperfect.

Step 3 · The PRS, in one line

What it does. Computes `prs = G @ weights`, the sum of dosage times weight over all variants, for every person at once, then standardizes it.

What you should see. One PRS value per person; the first few printed as standardized scores around zero.

The point. This is the whole equation from the lecture, vectorized. A matrix product is nothing more than the weighted sum written once for 4,000 people. Everything before and after this line is setup and interpretation.

Step 4 · Do cases score higher?

What it does. Plots the standardized PRS for controls and for cases on one axis.

What you should see. The case distribution shifted toward higher scores, with heavy overlap. You may also see a few clumps rather than one smooth curve: those are the 0 / 1 / 2 dosage groups of the large APOE-like variant, a sign one variant is doing a lot of the work.

The point. Separation is on average, not per person. A high score raises probability; plenty of controls still score high. This overlap is the single most important intuition about a PRS: it is risk, not a diagnosis.

Step 5 · Measure it: AUC and ROC

What it does. Computes the AUC and draws the ROC curve.

What you should see. An AUC of about 0.66.

The point. AUC is the chance a random case scores above a random control: 0.5 is a coin flip, 1.0 is perfect. About 0.66 is realistic, not a weak result. Common-disease PRS usually land in the 0.6s and earn their keep at the tails of the distribution, not in the middle.

Step 6 · Remove the biggest-effect variant

What it does. Drops the APOE-like variant, recomputes the PRS from the remaining nineteen, and compares the AUC.

What you should see. AUC with the variant about 0.66; without it about 0.54, almost back to a coin flip. Check it against the prediction you wrote down.

The point. One locus was carrying most of the signal. A score can be called polygenic and still depend heavily on a single variant, so the habit to build is to always report the score with and without the large-effect locus.

What maps to what

The real tools differ from these miniatures in degree, not in kind: real cohorts, shrinkage-based weights, and genuine validation. The behaviours you just watched are the real ones.

Your toy step	The real thing	What the real one adds	What it still does not fix
`synthetic cohort`	a GWAS cohort / biobank	millions of real people, real LD	ascertainment and selection biases
`downloaded weights`	PGS Catalog (PRS-CS, LDpred2)	shrinkage, an LD reference, validation	ancestry-specific: poor portability
`G @ weights`	plink --score, applied PGS	QC, missing genotypes, allele alignment	still only a weighted sum
`AUC / ROC`	validation metrics	large test sets, confidence intervals	modest ceiling for common disease
`leave-one-out`	the APOE in/out analysis	real effect sizes and loci	one locus can still dominate

Troubleshooting

Symptom	Likely cause and fix
Every cell looks like one long line	The notebook saved with broken line endings. Download it again from Canvas.
ModuleNotFoundError	You are not in the biot6900 environment. Activate it and restart the kernel.
Numbers differ from this guide	Run Kernel → Restart & Run All. Later cells depend on earlier ones, so running out of order silently changes results.
AUC is not about 0.66	You changed the random seed or the effect sizes. Reset them, or note the change and explain it.

The distribution shows clumps, not a smooth curve	Expected. The clumps are the dosage groups of the large APOE-like variant.
roc_auc_score raises an error	All individuals ended up one class. With the given seed this will not happen; re-run from the top.

What this lab deliberately leaves out

- No real genotypes, no real weights, no real disease labels. Every number came from a simulation you control.
- No ancestry and no portability. The whole cohort is one population; a single AUC tells the whole story. Part 2 adds ancestry, where that stops being true.
- No calibration against truth and no clinical covariates. The score is raw, and it is the only predictor in the room.

What comes next

Part 2 runs the real workflow on real 1000 Genomes genotypes with a published PGS Catalog score for your Part I disease. It adds the ancestry and portability problem (the AUC drop across population groups), calibration, and a clinical covariate, and it is graded. Keep this notebook: the reading habits transfer even though the data does not.